

MADL

Mobile Application Description Language

Reference Booklet v0.1

MADL is a minimal, precise vocabulary for describing any mobile application unambiguously — to yourself, to collaborators, and to AI coding agents. It is not a programming language. It is not a design system. It is a description language: a shared dictionary with enforced ID patterns.

Design principle: if two people read the same MADL description and disagree on what to build, the spec is wrong. MADL descriptions must be unambiguous.

1. Principles

#	Rule
01	Every node has a mandatory ID. No ID = does not exist in the spec.
02	IDs are cascading. The full ID encodes the complete path from app root.
03	IDs are stable. Once assigned, never change. Retired IDs are never reused.
04	IDs are lowercase, dot-separated, no spaces, no special characters.
05	Reuse WML/WAP terms where they fit. New terms only where WML has no equivalent.
06	Closed sets are closed. Adding a value requires a spec change, not a workaround.
07	Structure (what exists) is described separately from Behaviour (what happens).

2. The Five Baskets

MADL organises all concepts into five baskets. Read the layout as a system: centre holds the invariant core. Left side is static structure. Right side is dynamic behaviour. Top is close to the user. Bottom is close to the machine.

Position	Basket	Rationale
CENTRE	STRUCTURE	The invariant skeleton. Every other basket references this.
TOP-LEFT	NAVIGATION	Closest to user intent. Spatial movement through structure.
TOP-RIGHT	INTERACTION	What the user does on a card. Triggers, actions, transitions.
BOTTOM-LEFT	ELEMENTS	Components populating slots within cards. Static building blocks.
BOTTOM-RIGHT	SERVICES	Everything the app talks to outside its own structure.

3. Structure — the Centre

The addressable hierarchy of the application. All other baskets reference this.

Node types

Term	WML Origin	Definition
app	new	The root container. One per product.
deck	WML card parent	A full-viewport destination. One deck visible at a time.

<code>card</code>	WML	One named state of a deck. Same route, different appearance.
<code>layer</code>	new	Persistent chrome shared across decks (nav bar, tab bar). Not a deck.
<code>slot</code>	new	A named region within a card that accepts elements.

ID patterns

Term	Pattern	Example
<code>app</code>	<code>{app_id}</code>	<code>trk</code>
<code>deck</code>	<code>{app}.d{n}</code>	<code>trk.d2</code>
<code>card</code>	<code>{deck}.c{n}</code>	<code>trk.d2.c3</code>
<code>layer</code>	<code>{app}.l{n}</code>	<code>trk.l1</code>
<code>slot</code>	<code>{card}.s{n}</code>	<code>trk.d2.c3.s2</code>

Standard card names

Recommended identifiers for the condition a card represents. Use consistently.

Name	Condition
<code>default</code>	Normal loaded state. Use when no other name applies.
<code>empty</code>	No data to display. First-use or post-clear.
<code>loading</code>	Async operation in progress. Data not yet available.
<code>loaded</code>	Data present and rendered. Primary content state.
<code>error</code>	A failure occurred. Network, validation, or server.
<code>editing</code>	User is actively modifying existing data.
<code>saving</code>	Write operation in flight. Awaiting confirmation.
<code>success</code>	Operation completed successfully.

4. Navigation — Top-Left

How the user moves between decks and cards.

Terms

Term	WML Origin	Definition
<code>atlas</code>	new	Complete map of all decks, cards, and permitted transitions. One per app.
<code>stack</code>	new	Navigator that pushes decks onto history. Back returns to previous.
<code>tabset</code>	WML-adjacent	Navigator presenting N decks as parallel persistent destinations.
<code>drawer</code>	new	Navigator hidden off-screen, revealed by gesture or trigger.
<code>flow</code>	new	Named linear sequence of decks with declared entry and exit.
<code>entry</code>	WML	Default card shown when a deck is first opened.
<code>exit</code>	WML	Card or deck reached when user leaves via back/cancel/complete.

ID patterns

Term	Pattern	Example
<code>atlas</code>	<code>{app}.atlas</code>	<code>trk.atlas</code>
<code>stack</code>	<code>{app}.nav.stk{n}</code>	<code>trk.nav.stk1</code>
<code>tabset</code>	<code>{app}.nav.tab{n}</code>	<code>trk.nav.tab1</code>
<code>drawer</code>	<code>{app}.nav.drw{n}</code>	<code>trk.nav.drw1</code>

flow	<code>{app}.flow.{name}</code>	<code>trk.flow.onboard</code>
------	--------------------------------	-------------------------------

Atlas declaration format

```
{app}.atlas:
  {deck_id}:
    entry: {card_id}
    exit:  {deck_id | card_id}
    nav:   {navigator_id}
```

Topology selection

Condition	Use
User drills into detail, can go back	<code>stack</code>
User switches between top-level destinations	<code>tabset</code>
User accesses secondary / settings	<code>drawer</code>
User completes a linear multi-step task	<code>flow</code>
Overlay sequence within a screen	<code>modal-stack</code>

5. Interaction — Top-Right

What the user does and what happens as a result.

Terms

Term	WML Origin	Definition
<code>trigger</code>	WML	The gesture or system event that initiates an action.
<code>action</code>	WML type	A named operation bound to a trigger.
<code>transition</code>	new	Movement from one card or deck to another, caused by an action.
<code>guard</code>	new	A condition that must be true for a transition to execute.
<code>gesture</code>	new	The physical trigger type. Closed enumeration.

ID patterns

Term	Pattern	Example
<code>trigger</code>	<code>{card}.t{n}</code>	<code>trk.d2.c1.t1</code>
<code>action</code>	<code>{trigger}.a</code>	<code>trk.d2.c1.t1.a</code>

Gesture vocabulary — closed set

Gesture	Description
<code>tap</code>	Primary single-finger tap.
<code>long-press</code>	Sustained press, typically 500ms or more.
<code>swipe-left</code>	Horizontal swipe toward left edge.
<code>swipe-right</code>	Horizontal swipe toward right edge.
<code>swipe-up</code>	Vertical swipe toward top edge.
<code>swipe-down</code>	Vertical swipe toward bottom edge.
<code>pinch</code>	Two-finger pinch inward. Typically zoom-out or dismiss.
<code>expand</code>	Two-finger expand outward. Typically zoom-in.
<code>scroll-end</code>	User has reached the end of scrollable content.
<code>back</code>	Device or OS back action (hardware button or gesture).

system-event	Trigger fired by a service event, not a user gesture.
------------------------------	---

Transition types — closed set

Type	Description
push	New deck slides in from right. Adds to stack.
pop	Current deck slides out to right. Removes from stack.
replace	Current deck replaced with no stack change.
modal-present	Deck or overlay slides up from bottom. Interrupts flow.
modal-dismiss	Modal slides down. Returns to underlying content.
sheet-up	Partial-height overlay slides up from bottom.
sheet-down	Sheet dismisses downward.
fade	Cross-fade between source and target.
none	Instant switch. No animation.

Action declaration format

```
{trigger_id}:
  gesture:  {gesture_value}
  element:  {element_id}      # optional
  action:
    type:    {transition_type}
    target:  {card_id | deck_id}
    guard:   {condition}      # optional
    animation: {transition_type} # optional
```

6. Elements — Bottom-Left

The components that populate slots within cards.

Terms

Term	WML Origin	Definition
element	WML generic	Any discrete UI component placed in a slot.
field	WML	An element that accepts user input.
label	WML	An element displaying static or data-bound text.
control	new	An element that triggers an action (button, toggle, selector).
media	new	An element displaying image, video, or audio.
list	new	A repeating element rendering a collection of bound items.
overlay	new	An element rendered above the card surface. Always sub-typed.

ID patterns

Term	Pattern	Example
element	<code>{slot}.e{n}</code>	<code>trk.d2.c1.s2.e1</code>
overlay	<code>{card}.o{n}</code>	<code>trk.d2.c1.o1</code>

Overlay sub-types — closed set

Sub-type	Description	modal
dialog	Small centered modal. Confirmation, alert, short input.	always true
sheet	Slides up from bottom. Partial or full height.	declare explicitly

toast	Non-modal notification. Auto-dismisses.	always false
tooltip	Contextual info bubble attached to a specific element.	always false
action-sheet	List of contextual choices sliding up from bottom.	always true

Element declaration format

```
{element_id}:
  type:      {element_type}      # MANDATORY
  label:     {string}            # display text
  bound-to:  {endpoint | store}   # data source
  visible-if: {boolean_condition} # conditional render
  modal:     true | false        # overlays only
  auto-dismiss: {integer}         # toast only, seconds
  placeholder: {string}          # field only
```

7. Services — Bottom-Right

Everything the application communicates with outside its own structure.

Terms

Term	Definition
service	Any external system the app communicates with.
store	Local persistent data layer. On-device database, keychain, or preferences.
endpoint	A named operation exposed by a service.
binding	A declared link between a card/element and a store or endpoint.
host	The device operating system — camera, GPS, biometrics, notifications.
cloud	A remote platform service — auth, storage, push, analytics.
event	An async message arriving from a service that may trigger a card change.

ID patterns

Term	Pattern	Example
service	<code>{app}.svc.{name}</code>	<code>trk.svc.api</code>
store	<code>{app}.store.{name}</code>	<code>trk.store.db</code>
endpoint	<code>{service}.ep.{name}</code>	<code>trk.svc.api.ep.save</code>
host	<code>{app}.svc.host.{cap}</code>	<code>trk.svc.host.gps</code>
cloud	<code>{app}.svc.cloud.{name}</code>	<code>trk.svc.cloud.auth</code>
event	<code>{service}.evt.{name}</code>	<code>trk.svc.api.evt.sync</code>

Host capabilities — closed set

camera	gps	biometrics	notifications	contacts	accelerometer	microphone	nfc	bluetooth
--------	-----	------------	---------------	----------	---------------	------------	-----	-----------

Service declaration format

```
{app}.svc.{name}:
  type:      api | store | host | cloud
  protocol:  REST | GraphQL | WebSocket | native | SDK
  base-url:  {url}                # api and cloud only

{service}.ep.{name}:
  method:    GET | POST | PUT | DELETE | SUBSCRIBE
  path:      {path}
  input:     [{element_ids}]
  output:    {card_id | element_id}
```

```
on-error: {card_id}
```

8. Cascading ID Examples

Example 1 — Submit button triggering a save transition

trk	app: Tracker
trk.d2	deck 2: Data Entry
trk.d2.c1	card 1: default (form empty)
trk.d2.c1.s2	slot 2: footer
trk.d2.c1.s2.e1	element 1: Submit (type: control)
trk.d2.c1.t1	trigger 1: tap on Submit
trk.d2.c1.t1.a	action: push -> trk.d2.c3 if form.valid
trk.d2.c3	card 3: saving

Example 2 — GPS binding on a map screen

trk	app: Tracker
trk.d3	deck 3: Map View
trk.d3.c1	card 1: loaded
trk.d3.c1.s1	slot 1: main content
trk.d3.c1.s1.e1	element 1: map (type: media)
	bound-to: trk.svc.host.gps

Example 3 — Toast overlay triggered by async sync event

trk.svc.api	service: REST API
trk.svc.api.evt.sync	event: sync-complete
-> trk.d1.c1.o1	fires overlay 1 on deck 1 card 1
trk.d1.c1.o1	type: toast
	label: "Saved"
	modal: false
	auto-dismiss: 3

9. Complete ID Pattern Reference

{app}	trk
{app}.d{n}	trk.d2
{app}.d{n}.c{n}	trk.d2.c3
{app}.d{n}.c{n}.s{n}	trk.d2.c3.s1
{app}.d{n}.c{n}.s{n}.e{n}	trk.d2.c3.s1.e2
{app}.d{n}.c{n}.t{n}	trk.d2.c3.t1
{app}.d{n}.c{n}.t{n}.a	trk.d2.c3.t1.a
{app}.d{n}.c{n}.o{n}	trk.d2.c3.o1
{app}.l{n}	trk.l1
{app}.atlas	trk.atlas
{app}.nav.stk{n}	trk.nav.stk1
{app}.nav.tab{n}	trk.nav.tab1
{app}.nav.drw{n}	trk.nav.drw1
{app}.flow.{name}	trk.flow.onboard
{app}.svc.{name}	trk.svc.api
{app}.svc.host.{cap}	trk.svc.host.gps
{app}.svc.cloud.{name}	trk.svc.cloud.auth
{app}.store.{name}	trk.store.db
{app}.svc.{name}.ep.{name}	trk.svc.api.ep.save
{app}.svc.{name}.evt.{name}	trk.svc.api.evt.sync

10. Contested Terms — Adopted Definitions

card vs state vs screen

Disagreement: React Navigation: "screen" · SwiftUI: "view" · UX practitioners: "state" · WML: "card"

Adopted: card

Names the structural node, not the condition. "state" is reserved for runtime data values, not spec nodes. A deck has cards. A card IS a named configuration of its deck.

layer vs layout vs chrome

Disagreement: Expo Router: "layout" · React Navigation: "navigator wrapper" · Designers: "chrome" or "persistent UI"

Adopted: layer

Layout implies arrangement, chrome implies styling. A layer is a structural stratum that persists across deck transitions. Unambiguous in the OSI/network sense.

flow vs journey vs wizard

Disagreement: UX: "journey" · Product: "flow" · Developers: "wizard" for linear multi-step sequences

Adopted: flow

"Journey" is too poetic. "Wizard" implies a specific UI pattern. A flow is a named, directed sequence of decks with declared entry and exit. Platform-neutral.

trigger vs event vs interaction

Disagreement: Many frameworks conflate all three into a single concept

Adopted: three separate terms

trigger = user gesture or initiating condition. event = system/service async message. action = operation that executes in response. Conflating them is the root cause of most spec ambiguity.

overlay vs modal

Disagreement: "Modal" describes behaviour (blocks background), not shape. Many teams use "modal" as synonym for any overlay.

Adopted: overlay as structural term, modal as boolean property

overlay declares what the thing IS (rendered above card). modal:true|false declares how it BEHAVES. A sheet can be modal or non-modal. A dialog is always modal:true.

11. Summary Statistics

Metric	Value
Total terms defined	34
WML terms reused	6
New terms introduced	28
Closed enumerations	4
Contested definitions resolved	5
ID segment types	9
Baskets	5